

GUIDE UTILISATEUR

Arduinode

Auteur : Adam Bounour

Table des matières

I. Présentation du code.....	3
II. Présentation de l'interface.....	4
III. Comment récupérer les fichiers de fabrication pour imprimer le PCB ?.....	9
IV. Donner un nom aux Arduinodes.....	13

Avant de commencer, il est important de noter quelques points importants.

Tout d'abord, tous les codes, le projet KiCad ainsi que quelques informations complémentaires sont disponibles sur ce repo github :

<https://github.com/Adbo03/Node-sismique-3C.git>

Il est important de noter que ce projet a été développé sous Windows 11 et qu'il ne fonctionnera que sur cet OS, certaines fonctions et bibliothèques lui étant propres (DATA_COLLECT_WIFI.py). Ce projet peut être adapté sans grosses difficultés pour fonctionner sur d'autres OS mais cela ne sera pas traité ici.

De plus, pour la collecte WiFi des données de la carte SD, il est nécessaire d'activer la localisation. Cela peut facilement être fait dans les réglages de votre ordinateur. Sans cela, la collecte WiFi ne fonctionnera pas et un message d'erreur apparaîtra.

I. Présentation du code

Le projet repose sur 5 fichiers clés :

- ***ADXL_SPI.ino*** : Il s'agit du code des arduinodes, à téléverser sur les cartes. Il permet le paramétrage des arduinodes, la réception et l'envoi des données en BLE et en WiFi, ainsi que le basculement entre les différents modes de fonctionnement.
- ***SERVEUR_ADXL.py*** : Il s'agit du code de l'interface python pour visualiser en temps réel les données extraites de l'ADXL. Via cette interface, on peut aussi le paramétrer (fréquence d'échantillonnage, plage de mesure). Au-delà de l'ADXL, c'est l'arduino qu'on vient contrôler. La collecte et la conversion des données peuvent aussi être lancées depuis cette interface.
- ***CONVERSION_BIN_CSV.py*** : Il s'agit d'un code annexe utilisé par *SERVEUR_ADXL.py* afin de récupérer et de convertir les données à partir de la carte SD (la collecte en WiFi se fait via à un autre code).
- ***DATA_COLLECT_WIFI.py*** : Il s'agit d'un code annexe utilisé par *SERVEUR_ADXL.py* afin de collecter les données de la carte SD en WiFi. Cela évite de devoir manipuler la carte SD directement. Ce code introduit aussi la récupération des données de plusieurs arduinodes en une seule opération.
- ***SOURCE_BLE.ino*** : Il s'agit du code de la source. Avec ce code, la source peut générer des vibrations (signal ricker) à différentes fréquences. L'activation et le paramétrage de cette dernière se fait aussi depuis l'interface (code *SERVEUR_ADXL.py*).

II. Présentation de l'interface

On peut lancer l'interface de plusieurs façons :

- Avec l'application "ScanEarth" disponible dans le repo git. C'est la manière la plus simple et directe. Son utilisation est donc recommandée.

Rappel : comme mentionné, cette application ne fonctionnera que sur windows.

- Avec le terminal, en exécutant cette ligne de commande :

```
python ./SERVEUR_ADXL.py
```

Veillez à avoir tous les fichiers python dans le même répertoire.

Une fois l'application lancée, l'Interface Homme-Machine (IHM) apparaît avec un terminal. Le terminal permet d'avoir des informations de suivi. Il permet aussi d'afficher des messages d'erreurs pour d'éventuels problèmes de configuration.

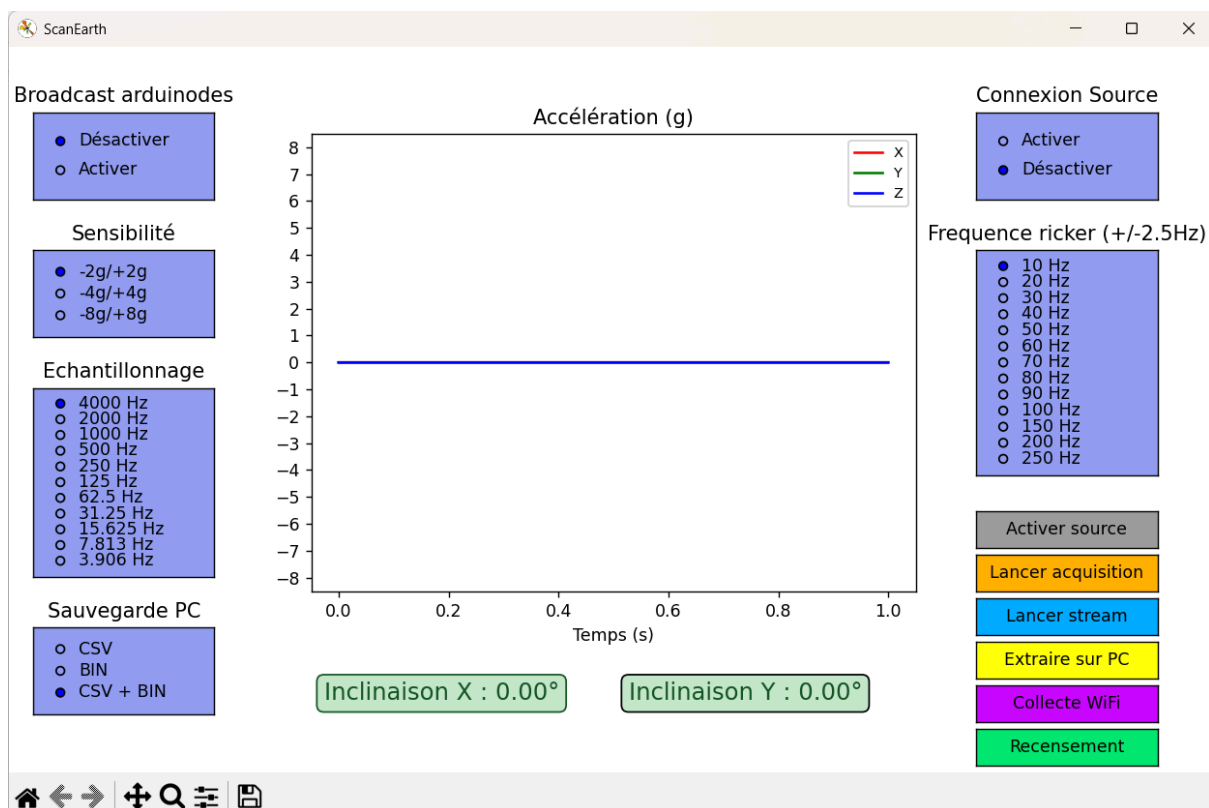


Image 1. Interface ScanEarth avant connexion

Comme le montre l'image 1, l'IHM de l'application comporte différents blocs et boutons qui ont tous un rôle bien précis.

- Bloc "Broadcast arduinodes" : Le système ainsi développé permet de configurer plusieurs arduinodes en même temps. En étant connecté à une arduinode, cette dernière peut propager sa configuration aux autres. Ce bloc permet d'activer ou non cette fonctionnalité. A noter que par défaut, le broadcast est activé.
- Bloc "Sensibilité" : Avec ce bloc, l'intervalle de mesures de l'ADXL peut être paramétré. Cela a une incidence sur la résolution spectrale de l'accéléromètre.
- Bloc "Echantillonnage" : La fréquence d'échantillonnage de l'accéléromètre peut être modifiée pour faire varier la précision du signal temporel.
- Bloc "Sauvegarde PC" : Il est possible de récupérer les données stockées dans la carte SD de deux manières différentes. La première consiste à connecter directement la carte SD à l'ordinateur. La deuxième utilise le WiFi de la carte. Peu importe la méthode choisie, ce bloc permet de choisir sous quel format on veut sauvegarder ces données. A noter que par défaut, on sauvegarde les données sous formats binaire et csv.
- Bloc "Connexion Source" : En fonction des configurations sur le terrain, on pourrait avoir envie d'utiliser la source développée dans le cadre de ce projet ou une autre source de vibrations. Ce bloc permet d'exprimer cette volonté en évitant à l'ordinateur de chercher à se connecter à une source inexistante. A noter que par défaut, la source est désactivée côté utilisateur.
- Bloc "Fréquence ricker (+/- 2.5 Hz)" : Si on choisit de connecter la source, on peut venir configurer la fréquence du ricker généré.
- Bouton "Activer source" : Permet d'activer ou de désactiver la source. Lorsque la source n'est pas connectée, le bouton reste grisé et inactif.
- Bouton "Lancer acquisition" : Ce bouton permet de lancer la sauvegarde des données de l'accéléromètre sur la carte SD. Lorsqu'une acquisition est en cours, il n'est plus possible de changer la plage de mesure ou la fréquence d'échantillonnage. Si on veut modifier ces paramètres, il faut arrêter l'acquisition en cours et en lancer une nouvelle. Cela évite de fausser les données.

- Bouton “Lancer stream” : Lorsqu’on allume les arduinodes, elles sont par défaut en mode Stand-by pour économiser de la batterie. Ce bouton permet de changer le mode de fonctionnement pour visualiser les données en temps réel sur le graphe au centre de l’IHM. Une fois ce mode sélectionné, on voit alors des courbes apparaître sur la fenêtre de visualisation et les deux voyants mesurant l’inclinaison de l’ADXL s’activer. A noter qu’on ne peut pas lancer une acquisition en même temps que le stream de données. Il s’agit de deux modes de fonctionnement distincts.
- Bouton “Extraire sur PC” : Après avoir configuré le mode de sauvegarde, ce bouton permet de récupérer les données à partir de la carte SD. Il faut s’assurer que la carte SD soit connectée à l’ordinateur.
- Bouton “Collecte WiFi” : Comme expliqué précédemment, ce bouton permet de collecter les données stockées dans la carte SD en WiFi. Cela évite de devoir récupérer les cartes SD de chaque arduinodes. Il se peut que le WiFi des cartes prenne un certain temps (quelques secondes) à être détecté. Si vous remarquez, lors d’une première collecte, que certaines cartes n’ont pas été détectées : pas de panique ! Il suffit de lancer une autre collecte WiFi pour récupérer les fichiers de données restants. En effet, que ce soit directement depuis la carte SD ou en WiFi, les données collectées sont automatiquement supprimées de la carte SD pour libérer le stockage. Ainsi, cela évite de collecter plusieurs fois les données d’une même arduinode.
- Bouton “Recensement” : Comme son nom l’indique, il permet de faire un scan du réseau pour obtenir le nombre d’arduinodes détectées.



```
[Recensement]      2 arduinodes détectées sur le terrain.
```

Image 2.1 Exemple de recensement (affichage console)

De plus, ce recensement permet de confirmer à chaque changement de configuration que toutes les arduinodes déployées ont bien été mises à jour.



```
[Synchronisation 2]  Arduinodes synchronisées !
```

Image 2.2 Message de synchronisation (affichage console)

Maintenant que toutes les sections de l'IHM ont été explicitement détaillées, il est important d'attendre que l'ordinateur soit connecté à l'arduino avant de vouloir la configurer. Pour ce faire, vous pouvez jeter un œil au terminal qui affichera toutes les étapes de connexion. Aussi, attendez que le nom de l'arduino apparaissent en haut de la fenêtre. C'est un indicateur sûr que l'arduino est bien connectée !

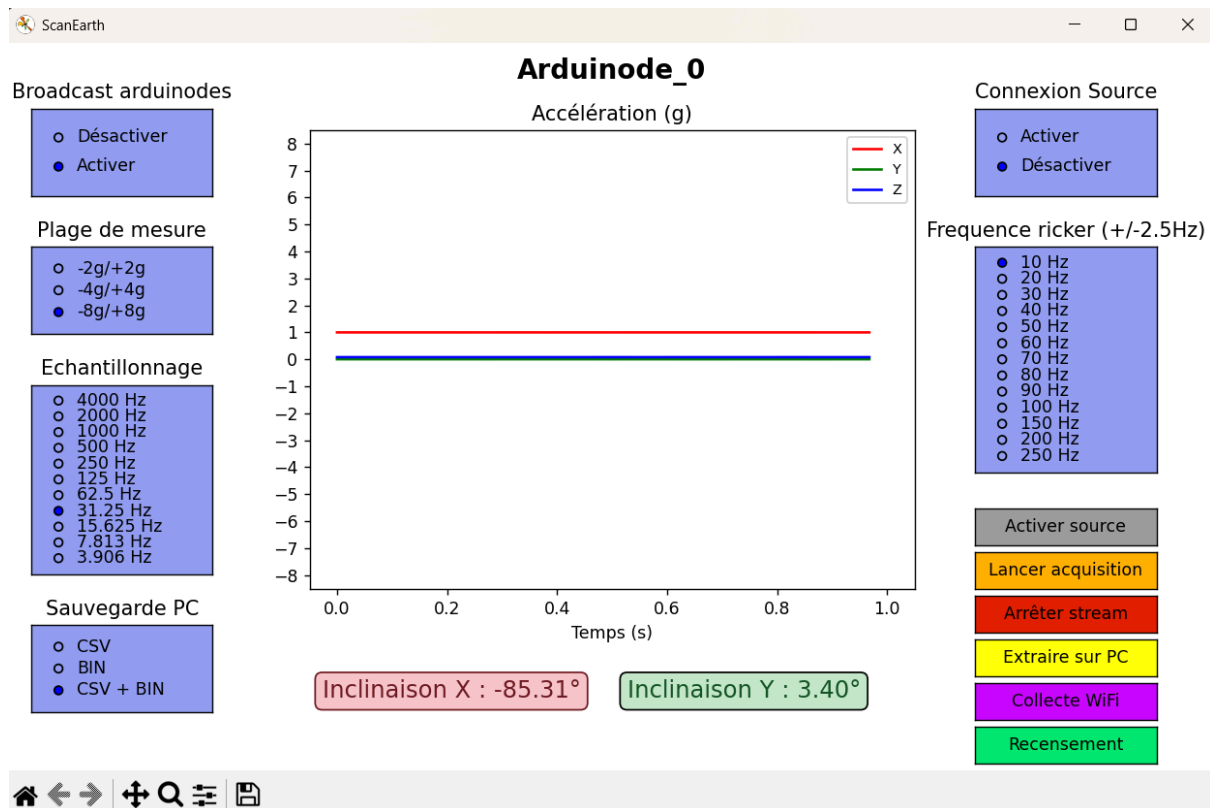


Image 3. Interface ScanEarth après connexion

Précision inclinaison :

Les voyants de l'inclinaison de l'ADXL se mettent au vert lorsque cette dernière est comprise dans un intervalle donné. Par défaut, l'inclinaison tolérée est de +/- 10°. Cette précision peut être modifiée dans le code SERVEUR_ADXL.py avec cette variable située au début du fichier (dans la section paramétrage) :

PRECISION_STABILITE = 10 # +/- exprimée en °

Ces voyants sont utiles pour savoir si l'ADXL est bien parallèle au sol. Sinon quoi, cela viendrait fausser les données collectées.

A noter que si cette variable est modifiée et que vous souhaitez utiliser l'application avec ces mises à jour, il faut régénérer l'exécutable de l'interface ScanEarth (plus d'informations sur le github).

Impulsions de la source:

Lorsque la source est connectée, un appui sur le bouton "Activer source" ou "Lancer acquisition" va générer une série d'impulsions. Le nombre d'impulsions générées lors d'un appui sur un de ces boutons peut être configuré.

```
#define MAX_NB_IMPULSE 3
```

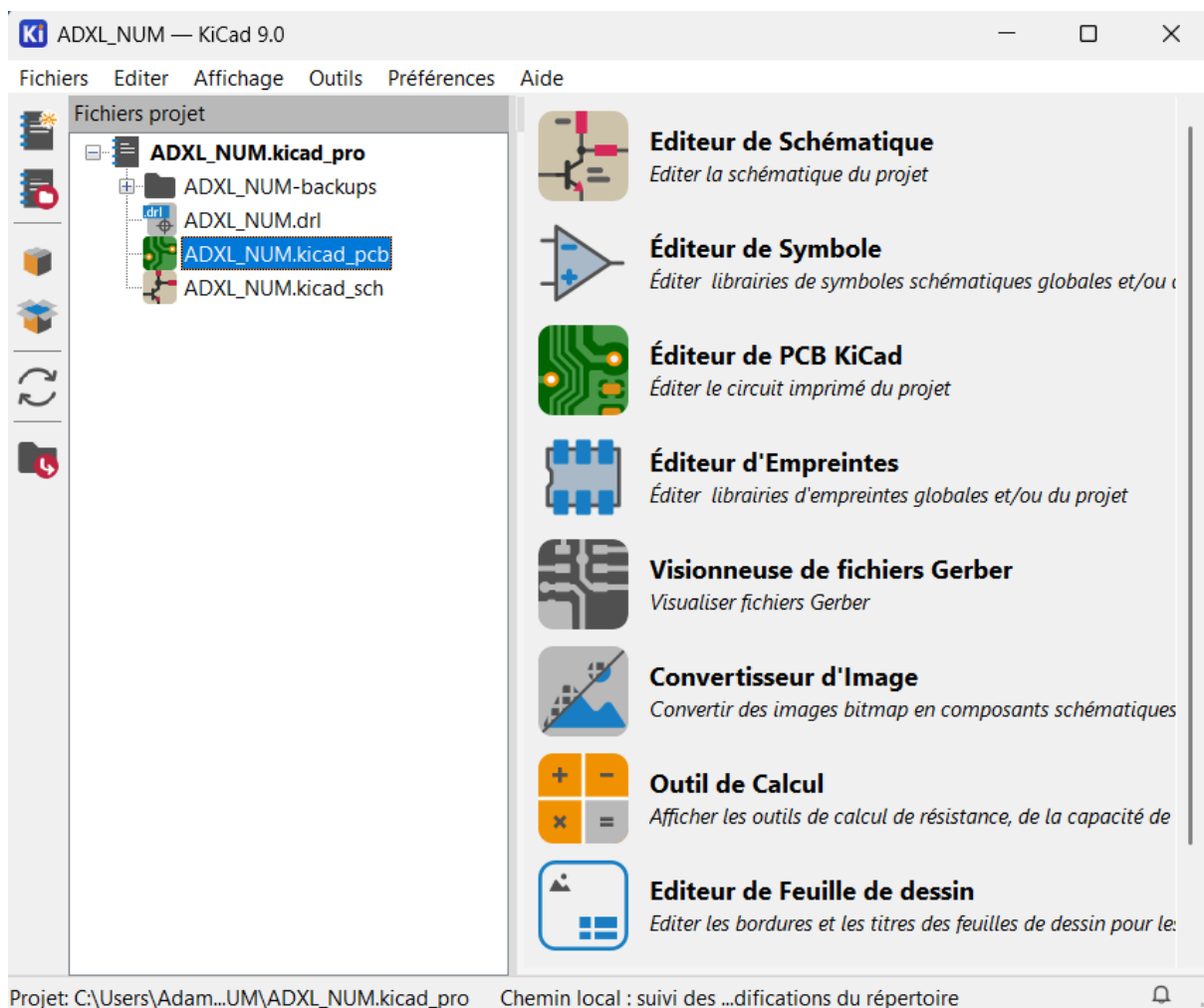
Dans le fichier SOURCE_BLE.ino, la constante MAX_NB_IMPULSE détermine le nombre d'impulsions désirées. En fonction de la configuration et de l'environnement dans lesquels les mesures sont réalisées, il peut être souhaitable de pouvoir modifier cette valeur. Cette constante, définie au début du fichier, permet exactement cela.

III. Comment récupérer les fichiers de fabrication pour imprimer le PCB ?

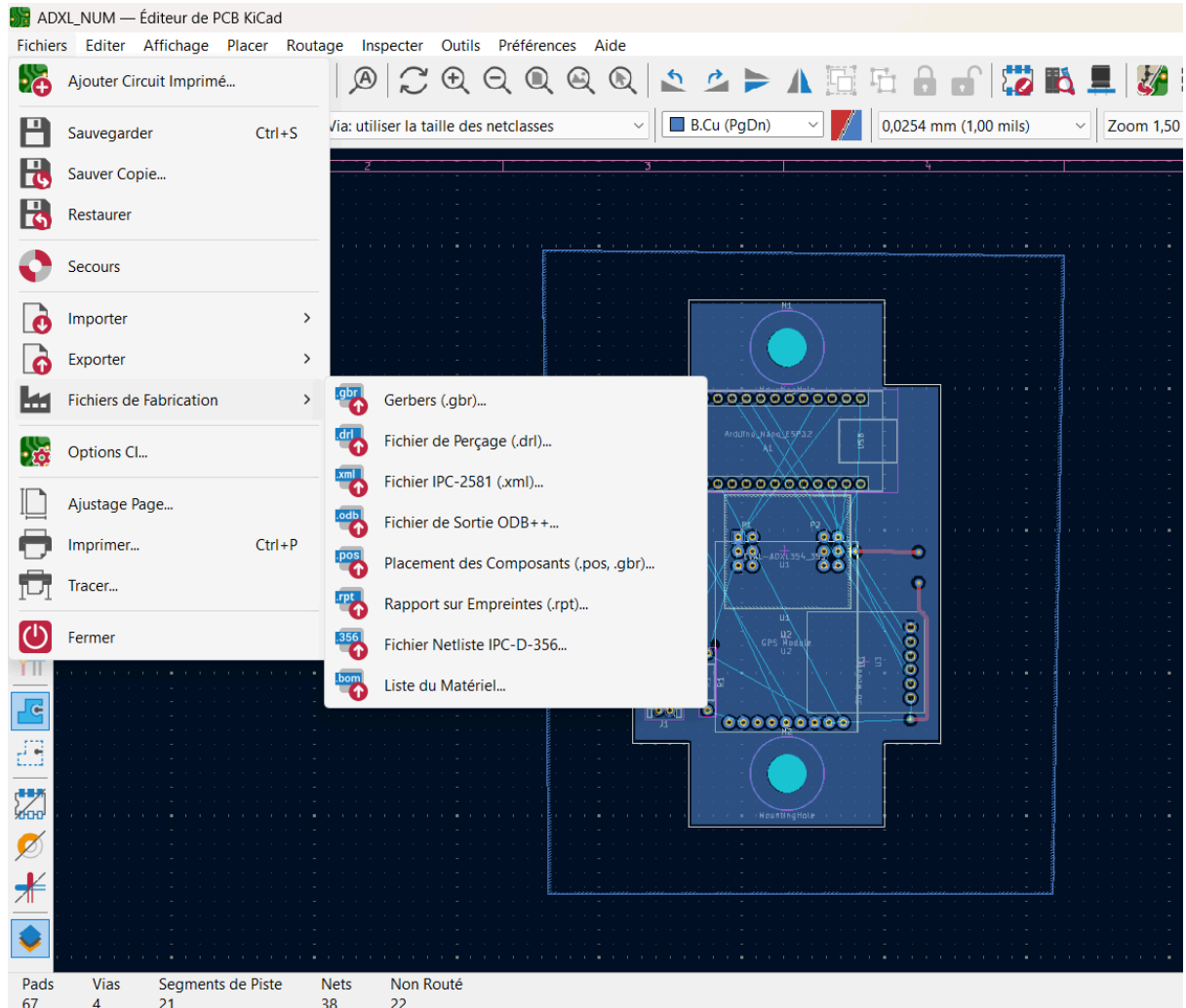
Les PCB pour ce projet ont été réalisés à l'aide de KiCad v9.0. Les projets KiCad sont disponibles sur le github.

Voici un guide étape par étape pour récupérer les fichiers gerbers ainsi que les fichiers de perçage pour l'impression.

1. Après avoir ouvert le projet KiCad, sélectionnez le fichier ADXL_NUM.kicad_pcb



2. Allez dans *Fichiers > Fichiers de fabrication*. Vous verrez plusieurs types de fichiers. Sélectionnez *Gerbers (.gbr)*



- Une nouvelle fenêtre s'ouvre alors. Laissez les configurations telles quelles et appuyer sur *Tracer*.

- Avant de refermer cette fenêtre, appuyer sur *Créer Fichiers de Perçage* en bas à droite de la fenêtre. Une autre fenêtre s'ouvre. Changez le format par *Gerber X2*. Cliquez ensuite sur *Générer*.

5. Il ne reste plus qu'à récupérer les fichiers de fabrications se trouvant dans *.ADXL_NUM_gers*.

IV. Donner un nom aux Arduinodes

Si vous voulez construire un réseau d'arduinodes, il est préférable de donner un nom à chaque module pour mieux l'identifier et analyser ses données. Dans cette optique, la fonction *ConfigureArduinodeID()* (ADXL_SPI.ino) permet exactement cela. Pour éviter de graver dans le code l'id de chaque arduinode et de devoir le modifier à chaque téléversement, cette fonction permet d'attribuer un identifiant dynamiquement.

```
void configureArduinodeID() {
    uint64_t chipID = ESP.getEfuseMac();
    Serial.printf("ID: %04X%08X\n", (uint16_t)(chipID >> 32), (uint32_t)chipID);

    // Utilisation de l'adresse MAC pour générer des identifiants uniques
    if (chipID == 0x4886AE63B0E4) { arduinodeID = 0; }
    else if (chipID == 0x98302E43CA48) { arduinodeID = 1; }
    else {
        arduinodeID = chipID;
    }

    nodeName = "Arduinode_" + String(arduinodeID);
    Serial.print("Module : "); Serial.println(nodeName);
}
```

Image 3. Fonction gérant l'attribution des noms (ADXL_SPI.ino)

Pour assurer un identifiant unique pour chaque arduinode, cette fonction utilise l'adresse MAC de la carte. Une modification de cette fonction est cependant requise à la création de chaque nouveau module. Il faut récupérer l'adresse MAC de la nouvelle arduinode et rajouter une autre condition pour lui attribuer l'identifiant souhaité. Cette opération n'est à réaliser qu'une seule fois et n'a pas besoin d'être répétée lors du déploiement des arduinodes sur le terrain. Si le code n'est pas modifié, l'arduinode fraîchement créée aura un identifiant qui dépendra du dernier octet de son adresse MAC. Cela risque de résulter en un réseau d'arduinodes aux noms aléatoires, ce qui peut compliquer par la suite le traitement des données collectées.

Voici un exemple qui illustre la procédure expliquée ci-dessus :

Pour cet exemple, nous allons téléverser le code sur une nouvelle arduinode. Nous supposons donc que l'adresse MAC de la carte n'est pas connue.

1. Téléversement du code pour la première fois

```
ID: 2C3CA0BD4D74
Module : Arduinode_48638906420596
ESP-NOW configuré et prêt pour la communication inter-arduinodes
Configuration BLE terminée.
Système prêt.
```

Lors du premier téléversement, voici ce qui apparaît sur le serial monitor d' Arduino IDE.

- ❖ Le champ "ID" nous donne un identifiant unique en hexadécimal basé sur l'adresse MAC de la carte
- ❖ Le champ "Module" affiche le nom donné à l'arduinode. Basé sur ID de la carte (en décimal cette fois), on remarque que ce nom n'a pas vraiment de sens. Il serait donc judicieux de le modifier pour obtenir un réseau de nodes aux noms homogènes.

2. Modification du nom de l'arduinode

```
void configureArduinodeID() {
    uint64_t chipID = ESP.getEfuseMac();
    Serial.printf("ID: %04X%08X\n", (uint16_t)(chipID >> 32), (uint32_t)chipID);

    // Utilisation de l'adresse MAC pour générer des identifiants uniques
    if (chipID == 0x4886AE63B0E4) { arduinodeID = 0; }
    else if (chipID == 0x98302E43CA48) { arduinodeID = 1; }
    else if (chipID == 0x2C3CA0BD4D74) { arduinodeID = 2; }
    else {
        arduinodeID = chipID;
    }

    nodeName = "Arduinode_" + String(arduinodeID);
    Serial.print("Module : "); Serial.println(nodeName);
}
```

Pour modifier le nom du module, il suffit de rajouter une condition "else if" dans la fonction de configuration. En prenant l'ID de la carte (affiché sur le serial monitor), on ajoute une condition pour changer manuellement le numéro du module.

3. Résultat

```
ID: 2C3CA0BD4D74  
Module : Arduinode_2  
ESP-NOW configuré et prêt pour la communication inter-arduinodes  
Configuration BLE terminée.  
Système prêt.
```

Après avoir modifié *configureArduinodeID*, on remarque que le nom du module a changé.